Advanced Tools for Complex Network Analysis
CNET 5052, Spring 2026

Prof. Brennan Klein and Dr. Milo Trujillo

Assignment 3, due by Friday, March 13, 2026 (a.o.e.)

Please submit a .pdf of your answers via Canvas, including a url to your .ipynb notebook in your fork of our class repository. Please name your Canvas submission as Lastname.pdf, (*e.g.* Klein.pdf).

AI Tags. Each question below has a tag indicating what kind of AI usage is appropriate, if at all:

- ✧ AI-Forbidden ✧ : Do not use generative AI for this. We want your own thinking and voice.
- ✧ AI-Limited ✧ : You may use AI for debugging help, documentation help, or short clarifications. Do not paste in complete solutions. You must still understand and be able to explain your work.
- ✧ AI-Permitted ✧ : You may use AI as a helper (including code suggestions), but you must cite it, verify by running and testing, and write your own explanations.
- ✧ AI-Encouraged ✧ : You are encouraged to use AI as a learning partner, if helpful. These parts are meant to build programming confidence and grasp.

In this course, the use of AI tools is experimental and intended to enhance your work and learning. Please use this privilege wisely, as we reserve the right to revoke this policy.

1. *Community detection, degree heterogeneity, and* `graph-tool` *(35 points).*

   ✧ AI-Encouraged ✧

   This question is a deeper dive into the inferential approach to network science, focusing specifically on the relationship between degree heterogeneity and block structure.

   (a) Using `graph-tool`, fit a degree-corrected and non-degree-corrected stochastic block model to the largest connected component of the `polblogs` dataset, and enforce that the resulting partitions only contain two blocks. Visualize the graph (twice) with the two different partitions. Hints: 1) Access this dataset using Netzschleuder, by calling `gt.collections.ns["polblogs"]`, 2) You can run then `g = gt.extract_largest_component(g, directed=False)`, and 3) The function `minimize_blockmodel_dl` takes multiple keyword arguments, which you will need to specify to complete this question.

   (b) Let's dive more into the Markov Chain Monte Carlo (MCMC) algorithm used to arrive at the partitions when fitting stochastic block models.

      i. Print the description length of the degree-corrected and non-degree-corrected models you fit in (b).

ii. Fit new degree-corrected and non-degree-corrected models to the largest component of the `polblogs` dataset. After initializing `BlockState` with `B=2`, use a `for` loop of 1,000 iterations to collect the description lengths of each model at every step using the `mcmc_sweep` function (setting `niter=1`; during the for-loop *do not enforce the number of blocks to be 2*). Visualize both of these curves in a single plot, labeled properly, and comment on your observation.

(c) Explore the **model selection** section in the `graph-tool` documentation and Section VIIA and VIIB in `https://arxiv.org/abs/1705.10225`. Figure 7 in this article should validate whether your approach in (b) above is on the right track. Summarize the intuition and formalism behind degree correction in the (microcanonical) stochastic block model framework. Use Figure 8 as your guide.

2. *Machine learning and a node-classification pipeline (20 points).*

✧ AI-Permitted ✧

This question is a direct extension of the workflow in `class_07_ml.ipynb`: build features, split train/test, fit a classifier, evaluate, and interpret results. Using the Zachary Karate Club graph from G = `nx.karate_club_graph()`. Each node has a ground-truth label `club` (either `"Mr. Hi"` or `"Officer"`). Treat this as your target variable y.

(a) Create a node-level feature matrix X with at least six features total:

- Four structural features computed from the graph (choose from: degree, clustering coefficient, PageRank, betweenness, eigenvector centrality, etc.).
- Two synthetic "non-network" attributes that you generate yourself (e.g., `attendance`, `tenure`, `height`, etc.), by sampling from different Gaussians *conditional on the club label* (state the means/standard deviations you used).

Briefly state (in one short paragraph) what your features are.

(b) Random Forest model.

i. Perform $R = 50$ different 80/20 train/test splits with stratification by y (use 50 different random seeds).

ii. For each split, fit a `RandomForestClassifier` (you may keep hyperparameters fixed across splits).

iii. Report the mean and standard deviation of test-set accuracy and macro-F1 across the R splits.

iv. For *one* representative split (choose a seed and state it), show the confusion matrix (table or plot).

(c) Report the top 5 features by importance and interpret what the model seems to be using to separate the two labels. You may use either: impurity-based importances (default in scikit-learn), or permutation importance.

(d) Robustness test: Refit the model under one of the following changes, and report the new test accuracy and macro-F1. Either:

i. Increase the noise (i.e., standard deviations) in your synthetic attributes, or

ii. Remove two of your structural features and refit, or

iii. Change one key random forest hyperparameter—for example, (e.g., `max_depth` or `min_samples_leaf`—and justify your choice.

Explain what changed and why you think it changed.

3. *Properties of random geometric graphs (25 points).*

✧ AI-Limited ✧

In class we discussed the *hard* random geometric graph (RGG): nodes connect if they are within a distance threshold. Many real systems are better described by a *soft* distance-decay mechanism, where long edges are possible but less likely. Throughout this problem, you will work with the logistic distance-decay model

$$p_{ij} = \sigma(\alpha - \beta d_{ij}), \qquad \sigma(z) = \frac{1}{1 + e^{-z}},$$

where d_{ij} is the Euclidean distance between nodes i and j in space, $\alpha \in \mathbb{R}$ controls baseline edge density, and $\beta \geq 0$ controls how strongly distance suppresses edges.

Unless otherwise stated, sample $N = 300$ node positions i.i.d. uniformly from the unit square $[0, 1]^2$, compute all pairwise distances d_{ij}, and consider undirected simple graphs with $A_{ii} = 0$ and $A_{ij} = A_{ji}$.

(a) Make a visualization that helps explain the effects and roles of α and β in this model. What happens when $\beta = 0$?

(b) Write a function that samples soft random geometric graphs, according to that connection rule above. Hint: Your function should: (i) compute d_{ij} for all $i < j$,(ii) convert to probabilities p_{ij}, and (iii) sample independent Bernoulli edges.

(c) Using $N = 300$ nodes in $[0, 1]^2$, sample two graphs with the same α but two different values of β (one small, one large). For each graph, report:

$$m = \sum_{i<j} A_{ij}, \qquad \langle k \rangle = \frac{2m}{n}, \qquad \langle d_{\text{edge}} \rangle = \frac{1}{m} \sum_{i<j} A_{ij} d_{ij}.$$

Then, visualize each network.

(d) Briefly describe how changing β affected the typical edge length of the graph.

4. *Spatial (road) networks with `osmnx` (20 points).*

✧ AI-Permitted ✧

In this problem, you will use `osmnx` to download and analyze drivable road networks from OpenStreetMap. You will (i) visualize and qualitatively compare two cities, (ii) design a quantitative dissimilarity measure between road networks, and (iii) scale this up to a fixed set of U.S. cities.

Reproducibility constraints (apply to every city):

- Use the same extraction rule for every city: a radius of 3219 meters (2 miles) around a single center point, and `network_type="drive"`.
- OSMnx returns a directed multigraph by default. For comparability, convert every graph to the same representation *before* computing features. You may choose either: (i) a directed multigraph representation, or (ii) an undirected simple-graph representation. State your choice once and use it consistently throughout.
- If your features depend on metric lengths/areas, project the graph first (e.g., `ox.project_graph`).
- Because repeated downloads can fail intermittently, you should cache graphs to disk (e.g., save GraphML) and reload them when needed.

(a) Pick a city that is personally meaningful to you (e.g., your hometown, a favorite place, somewhere you have lived). Using `osmnx`:

  i. Choose and clearly state a single center point for your city (e.g., "City Hall", "Downtown", or an address/landmark). Use this as the center of your query.
  ii. Load the drivable road network within a radius of 3219 meters of that point.
  iii. Create a visualization of the network, with a title/caption indicating the city, center point, and radius used. +5 bonus points will be awarded to the best visualization in the class.
  iv. Report basic size statistics: number of nodes N and edges M (under your chosen graph representation).

(b) Repeat part (a) for the coordinate: (32.335660407, −110.97908793) (Tucson, Arizona), using the same radius, network type, and graph representation as above. Report N and M.

(c) Provide a qualitative comparison between your city and Tucson based on your visualizations. Include at least five concrete observations.

(d) Design a function $D(G, H)$ that outputs a single number representing the dissimilarity between two road networks G and H. Your measure must satisfy:

$$D(G, H) \geq 0, \qquad D(G, H) = D(H, G), \qquad D(G, G) = 0, \tag{1}$$

and it must be comparable across networks of different sizes. (This question is asking you to design and implement a graph distance measure.)

(e) Justify your design choices: why should your features capture meaningful differences in road network structure? Your justification should also explain why your measure is size-comparable (e.g., normalization, standardization, or distribution-based distances).

(f) Compute $D(G_{\text{yours}}, G_{\text{tucson}})$ using your implementation. Report the value and interpret it in a few sentences in light of your qualitative observations.

(g) Boeing (2019)[1] analyzes 100 cities and visualizes their street orientation histograms in Figure 4. Using a subset of those cities (below), download each city's

[1] Boeing, G. (2019). "Urban Spatial Order: Street Network Orientation, Configuration, and Entropy." *Applied Network Science*, 4(1), 67.

drivable road network using `osmnx`. Using your graph distance measure, measure the pairwise dissimilarity between each of the networks, and put these distance values into a distance matrix. Visualize this matrix with `plt.imshow()`.

i. `"Atlanta, Georgia, USA"`
ii. `"Boston, MA, USA"`
iii. `"Buffalo, NY, USA"`
iv. `"Charlotte, NC, USA"`
v. `"Chicago, IL, USA"`
vi. `"Cleveland, OH, USA"`
vii. `"Dallas, TX, USA"`
viii. `"Houston, TX, USA"`
ix. `"Denver, CO, USA"`
x. `"Detroit, MI, USA"`
xi. `"Las Vegas, NV, USA"`
xii. `"Los Angeles, CA, USA"`
xiii. `"Manhattan, NYC, NY, USA"`
xiv. `"Miami, FL, USA"`
xv. `"Minneapolis, MN, USA"`
xvi. `"Orlando, FL, USA"`
xvii. `"Philadelphia, PA, USA"`
xviii. `"Phoenix, AZ, USA"`
xix. `"Portland, OR, USA"`
xx. `"Sacramento, CA, USA"`
xxi. `"San Francisco, CA, USA"`
xxii. `"Seattle, WA, USA"`
xxiii. `"St.  Louis, MO, USA"`
xxiv. `"Tampa, FL, USA"`
xxv. `"Washington, District of Columbia, USA"`

Practical note: If any city string fails to geocode reliably, you may manually disambiguate it (e.g., by adding a neighborhood/country) but you must document what you changed.